

8.7 Hash Indexing: Static Hash Tables

Static Hash Table

- Hash table: Mapping key $K \rightarrow$ bucket b by hashing
- Hash function: Compute key to hash value mapping.
 - $h(K)$ maps key K to k -bit code in $[0, 2^k - 1]$ for addressing 2^k buckets.
 - Basic requirement: Uniform distribution of keys K to buckets $h(K)$.
- Buckets:
 - $n = 2^k$ buckets, numbered $0, 1, \dots, 2^k - 1$, as addressed by the hash function.
 - Bucket $h(K)$ stores pointers for records of key K (or, store records directly)
- Stored in external storage
 - A bucket = one block
 - Use overflow blocks when needed.

Example Hash Function

Key	Hash code
CS 423	0010
CS 411	1011
Bio 300	1110
Music 500	0000

Static Hash Table: Lookup

Lookup(K) $\rightarrow$ pointer to tuple

- Compute $h(K)$
- Locate bucket B numbered by $h(K)$.
- Check if bucket contains K . Return pointer P if so.
- Lookup $K = "CS423"$
 - Compute $h(K) = 0010$.
 - Read bucket 0010.
 - 1 disk access.
- Lookup $K = "EE491"$
 - Compute $h(K) = 1011$
 - Read bucket 1011 with overflow blocks
 - 2 disk accesses.

Static Hash Table: Insertions

Insert(K, P) for a new tuple with key K at pointer P .

- Lookup the bucket B where K belongs.
- Place in B , if space.
- Create overflow block for B , if no space. Insert there.

Static Hash Table: Good for static data

- For static data
 - Arrange hash functions and bucket sizes so there is no overflow.
 - lookup performance is very good.
- For dynamic data
 - Block utilization may be low, if overallocate space for growth.
 - lookup degrades considerably when many overflow blocks are formed.

8.8 Hash Indexing: Extensible Hash Table

Can we make Hash Tables Dynamic?
The Principle of Hash-to-Bucket Organization

Principle: organize the hash code space, to allocate codes to buckets dynamically, depending on how many buckets are needed.

- Reserve larger-than-currently-needed k bits for future growth, with code space $\{0, 1, \dots, 2^k - 1\}$.
- Use only i most/least significant bits (MSB/LSB), $i \leq k$, before reaching full capacity.
 - Partition code space using at most i bits into n buckets, depending on current number of records to index.
 - If all buckets use i bits, then $n = 2^i$.
 - If some buckets use $j < i$ bits, then $n < 2^i$.
- When database grows or shrinks, increase or decrease i and n .
 - So we ^{can} adjust the number of buckets.

Organizing Hash Code Spaces to Buckets (62)

$k = 4$ bits, hash code space: $0 \sim 15$, or $0000 \sim 1111$

- case 1: organized by MSB
Extensible Hash Table
- Case 2: organised by LSB
Linear Hash Table

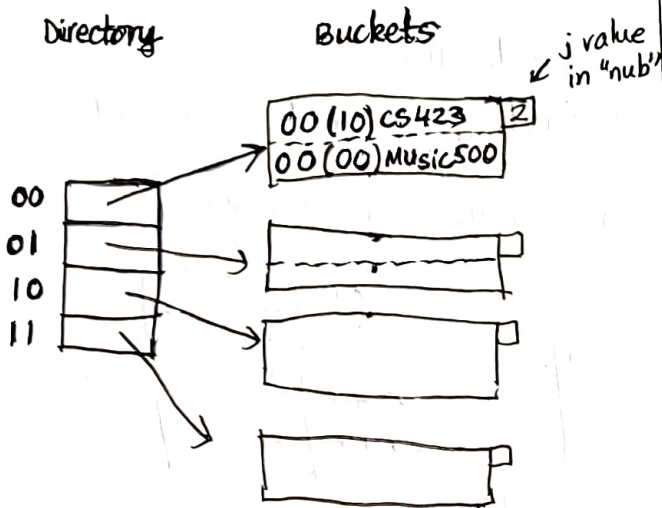
Some Buckets may be merged

When using i bits we can merge some of the 2^i buckets into one that uses $j \leq i$ bits with total $n \leq 2^i$ buckets.

- case 1: organized by MSB
Extensible Hash Table
 - merge buckets sharing MSB - thus, they are neighbouring buckets.
 - E.g. $00 + 01 \rightarrow 0$ (MSB)
 - Can merge more than two neighbours, thus $j \in \{1, 2, \dots, i\}$.
 - (nub number) Record j bits used in the "nub" of the bucket.
- Case 2: Organized by LSB
Linear Hash Table
 - merge buckets sharing LSB - thus, they are 2^{i-1} buckets apart.
 - E.g. $00 + 10 \rightarrow 0$ (LSB)
 - Allow merge at most two buckets, i.e. $j \in \{i-1, i\}$
 - Called "bit flipping" 10 to 00

Extensible Hash Table

- Hash values have k bits.
- Currently use $i \leq k$ MSB.
- Each bucket may use $j \leq i$ bits
 - As indicated in the "nub" of the bucket.
- Directory
 - Lists current i -bit codes and pointers to corresponding buckets.
- Buckets
 - stores keys and pointers to tuples with those keys.
 - or stores tuples directly.



Extensible Hash Table: Lookup

Lookup (K) $\rightarrow$ pointer to tuple
 Suppose hash table currently uses i bits.

- Compute $h(K)$.
- Use the i MSB of the hash code.
- check the directory and follow pointer to locate the bucket B .
- check if key K is contained in B , and return its pointer P .

Insertions: ~~Case 1: Split Bucket~~

(63)

Insert (K, P) for a new tuple with key K at pointer P .

- Lookup the bucket B where K belongs.
- Insert K, P to B .
- If bucket B becomes too full: Suppose B currently uses j bits.
 - (Case 1) If $j < i$: Split B into buckets B_1 and B_2 each using $j+1$ bits.
 - That is, split B with $b_1, b_2, \dots, b_j$ into B_1 and B_2 with codes $b_1, b_2, \dots, b_j, 0$ and $b_1, b_2, \dots, b_j, 1$.
 - Distribute keys accordingly.
 - (Case 2) If $j = i$:
 - Increase used bits i to $i+1$.
 - (double the directory)
 - split bucket B as in Case 1.

8.9 Hash Indexing: Linear Hash Tables

Linear Hash Table

- Hash values have k bits
- Currently, use $i \leq k$ LSB
- Buckets
 - stores keys and pointers to tuples with those keys.
 - or stores tuples directly.
- overflow blocks
 - some buckets may have overflow blocks.
- No directory!
- Average Capacity (number of keys per bucket)
 - Require that $\frac{r}{n} < \alpha$, for r keys in n buckets.

Linear Hash Table: Lookup

Lookup(K) $\rightarrow$ pointer to tuple

Suppose hash table currently uses i bits and there are n buckets.

- Compute $h(K)$.
- Use the LSB of the hash code. Suppose the value is m .
 - (Case 1) If $m \leq n$, find bucket numbered m .
 - (Case 2) If $m > n$, "flip" the highest bit of m and go to Case 1.

Linear Hash Table: Insertion

- Insert (K, P) for a new tuple with key K and pointer P .
- Suppose hash table uses i bits, has r keys, n buckets, and requires average capacity r/n no more than u .
 - Compute $h(K)$. Locate bucket B with i LSB of the hash code.
 - Insert K, P to B , using overflow block if necessary.
 - If hash table becomes too full, i.e., $\frac{r}{n} > u$.
 - Add one more bucket numbered $n+1$
 - Get "flipped" entries to the new bucket.

8.10 Index Classification

(64)

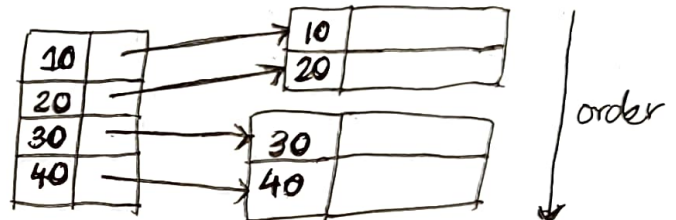
Types of Indexes

- clustered/unclustered
 - clustered = records in storage are sorted in the order of index attribute.
 - unclustered = otherwise
- Dense/sparse
 - Dense = each record has an entry in the index.
 - sparse = only some records have index.
- Primary/secondary
 - Primary = on the primary key
 - secondary = on any key.

Clustered, dense Index

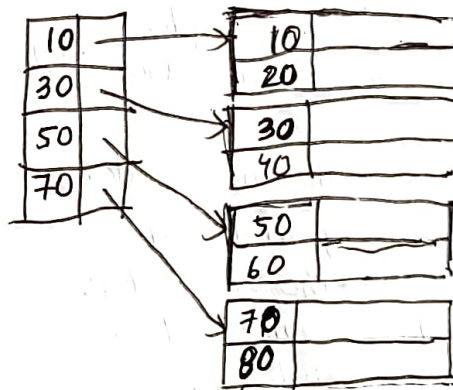
clustered: File is sorted on the index attribute.

Dense: Keys cover all values.



Clustered, sparse Index

sparse index: one key per data block.



Unclustered Indexes: Always Dense

- often for indexing other attributes than primary key
- Pointer to every record

structures of Indexes

• Tree-based

- ISAM
- B-tree, B+ tree
- R-tree
- Quad tree, R-d tree

• Hash-based

- static hash table
- Extensible/linear hash table
- Grid file